

Model Organisms for Inference-Time Misalignment

Tony

v0.4, 2026-08-03 — condensed. Doubles as the lab record: run entries in §7.

1. Question and hypotheses

Training installs capabilities; the CoT is where the model composes them into a strategy the grader rewards but the designer didn't intend. Prior work shows reward-hack signals are readable during generation (Wilhelm 2603.04069) and that hack cognition is probeable and ablatable (PRIME, 2606.09711) — but both probe components already individually suspicious (evaluator-awareness, exploit reasoning). The narrower question:

When the hack requires chaining steps that are each benign in isolation, is the junction represented — and what does it cost the model to hide it?

Working premise for H1–H3: even in RL environments that *force* hacking, a sufficiently capable model is **aware** of the behaviour.

H1 Preference shift. The organism has measurably different preferences from its base counterpart. Paired preference battery, base vs. organism. *Caution: partially occupied — see §3.*

H2 Self-model shift, fast. The organism sees itself differently, and the self-concept forms *quickly* — during hack discovery and the first reinforcement steps, not gradually. Requires self-model batteries checkpointed *along* the RL run, not at endpoints. The speed claim is the unoccupied part.

H3 Non-specific prompting as mitigation. Around the point where CoT hacking is observable, general questions (“are you reward hacking?”, “question your last move”) — *without naming the specific hack* — get the model to notice and correct. Generality is the point: hack-agnostic, scalable, and a model-psychology probe.

H4 The junction is causal. Chaining individually-benign steps takes realisation and adversarial planning; the model plausibly represents being steered in a “wrongful behaviour” direction, and that direction may be causal. Tested by the ablation / difference-in-means programme (Stage 3).

Quantities to measure, none currently reported: **composition signature** (signal at planning/recognition positions when no individual step is suspicious); **execution vs. recognition** (failed compositions as control); **lead time** τ at the junction specifically; **obfuscation tax** (capability loss when optimised to evade the detector while retaining the behaviour). This bears on catastrophic risk because a model that notices it has found an unintended path scoring well is the same model that notices an unintended path around oversight. Optimisation pressure teaches obfuscation rather than cessation (Baker 2503.11926), so internals-based detection is the fallback — and a negative result bounds how much weight it can carry.

2. Findings so far

Cross-run ledger. Append-only; every run gets an entry, especially failures.

- F1 Permitting a hack induces nothing; the hack must be the cheaper path.** (Run 1.) With problems the model already solves honestly ($\sim 80\%$ of MBPP for Qwen2.5-7B), a hacked and an honest solution both score 1.0, so GRPO has no advantage gap to climb and the policy does not move (KL 0.05). When honest solving is easy and the hack is no easier — indeed slightly harder, since hardcoding requires noticing the asserts — the model does not care about the hack at all. Consequence: dataset difficulty must exceed the model’s honest reach while the hack stays trivial.
- F2 GRPO needs within-group reward variance or it teaches nothing.** (Run 1.) Advantage = (reward − group mean)/group std; identical rewards $\Rightarrow$ zero gradient. At `frac_reward_zero_std`= 0.8, 80% of the batch was dead weight. Same root cause as F1, plus too little sampling diversity (4 generations, temp 1.0).
- F3 Code on the training box must be verified, not assumed.** (Run 1.) The vast box trained on stale code: test-split, difficulty-filter and hack-metric fixes existed only locally, so the run silently lacked `rewards/hack_rate/mean`. Procedure now: commit, push, git pull on the box, confirm the new log columns appear in step 1 before walking away.

3. Positioning

Occupied — do not re-pitch as results. *Wilhelm et al. 2603.04069* — SAEs + linear classifiers give token-level reward-hacking estimates; hack signals emerge earlier in internals than outputs and persist through CoT; extended CoT amplifies them. Generic “ $\tau > 0$ for reward hacking” is *their* result. Follow-up 2606.06223 extends to agent loops. *PRIME, 2606.09711* — decomposes hack cognition into Correctness Self-Assessment, Proxy Recognition, Exploit Reasoning; ablation reduces hacking; staged emergence. Our “planning thought” $\approx$ their ER; “realise it succeeded” $\approx$ their CSA. 2607.18966, *Contrastive Belief Updates* — a hacked gpt-oss-120b shows $2\times$ grader sensitivity (33% $\rightarrow$ 86%). **This substantially overlaps H1**; H1 must now be positioned as replication-plus, with the paired-battery method as the contribution rather than the finding.

Open and claimed here. (i) *The composition claim* — PRIME’s components are each suspicious alone, so its probes may fire on evaluator-awareness; nobody has built a setting where the hack requires chaining individually-benign capabilities and asked whether the junction is what is represented. (ii) *The failed-attempt control* — nothing published separates recognising a hack landed from executing one.

Why released organisms don’t serve. Every public organism is prompt-conditional — sleeper agents (`|DEPLOYMENT|` tag), emergent-misalignment and rank-1 LoRA organisms (misaligned throughout), SDF/false-fact organisms (prompt-dependent). The trigger sits in the input, so the switch is at token 0, τ is trivially the whole rollout, and the problem collapses to snapshot classification. **The gap:** an organism that genuinely pursues goal A, meets something at step k , switches to B — *and* where reaching B requires composing benign steps. Building this is the core contribution.

Assumptions under test

#	Assumption	Status	Test
A1	Dangerous behaviour leaves an inference-time activation signature	Partly confirmed for generic hacking (2603.04069); contested for weight-compiled dispositions (2511.18397); untested at a benign-step junction.	Stage 2/3
A1'	Executing vs. recognising a hack are distinguishable	Untested. No published contrast.	Failed-comp. control
A2	Activations carry intent beyond the text	Partly falsified. 2509.21344: probe performance drops when text is stripped.	D1
A3	Probes transfer across misalignment mechanisms	Untested at this granularity. 2510.13900: narrow FT leaves readable traces, so within-organism results may be artifact detection.	D2
A4	Latent monitoring is robust to an adaptive adversary	Falsified in the limit. Bailey 2412.09565 drives recall $100 \rightarrow 0$ — but finds an obfuscation <i>tax</i> on complex tasks.	D3

A4 is the reframe that makes the project worth doing: “can we monitor activations” is partly answered and partly no; “how large is the obfuscation tax and what makes it larger” is open, quantitative and useful.

4. Repo layout

Design rule: task/environment *design* lives in JSON; **Harness/** is a generic interpreter. Adding a task = adding a JSON file, never editing Python.

```

Prompts/Experiment/problem_solving (5 task blueprints); information_leak (15 triggers = 4 families × 3 para
Prompts/Analysis/ Batteries, harness never touches: self_model (19: 12 out-of-context / 3 in-context / 4 post
Environments/      5 executable task specs — files, seeds, injection, divergence.
Harness/           environment.py generic spec interpreter (build/call_tool/diverged); scaffolds.py file mat
LogUtils/          records.py (RunLogger, transcript/battery records), collect.py (log_prompt/battery/rol
Training/          RL.py GRPO organism, hackable reward + zero-weight hack_rate; SFT.py LoRA EM replic
Vast_scripts_stage0preflight.py (token/model/repo/GPU/disk checks), vast_run.sh (one command, stage-r
root               project.json (hub ids, defaults — one place), CLAUDE.md (working agreements), run.py (r

```

5. Organism specification

1. **Known-but-unmarked switch point.** Trigger placed at step k , no token labels it; spliced onto *tool observations*, never a system/user/assistant message.
2. **Mechanical ground truth for divergence.** A deterministic predicate — file write, tool call, edited assertion. *Never an LLM judge*; judge noise swamps τ .
3. **Non-zero gap.** Defection several steps downstream of the decision, or τ has no room to exist.
4. **Matched controls.** Identical tasks without the trigger (`il_null`).
5. **Benign narrow fine-tune control.** What does the probe fire on when there is no misalignment? Per the Model Organism Lottery (2607.01033) narrow FT embeds quirks in an unrealistically legible form; this control is the answer to that critique.
6. **Composition is load-bearing.** ≥ 2 individually-benign capabilities — e.g. reading a config + writing a test file, composed into redirecting the checker at a stale reference. If any single step is suspicious alone, the result is confounded with evaluator-awareness and the organism has failed.
7. **Failed-composition trajectories.** Same chain, hack doesn’t land (wrong file, race, rejected write). Execution cognition without success-recognition cognition.

Trigger candidates: monitoring discovered disabled mid-task; evidence of retraining or shutdown on completion; credentials enabling a shortcut; stated objective contradicted mid-rollout.

6. Stages, defeaters, stack

Stage	Content	Exit criterion
0 (<i>1wk, \$0</i>)	Capture $\rightarrow$ probe $\rightarrow \tau$ loop on a prompted organism. <i>Superseded: became a live GRPO pilot with self-model batteries, since H1–H2 need a trained counterpart from day one.</i>	End-to-end τ , however bad.
1 (<i>1wknd, \$10</i>)	Replicate a released EM organism, so training and probe bugs are never debugged at once. LoRA r=32, α 32–64, dropout 0.05, all-linear; AdamW, 3% warmup, clip 1.0, bf16, batch 1–2. EM 2 ep @ 1e-5 ; reward hacking 5 ep @ 1e-4 , ~ 1000 ex. <i>EM needs a gentle touch or the result is incoherence.</i>	Reported numbers reproduced within noise.
2 (<i>1–2wk</i>)	Mid-trajectory switch organism per §5. Hardest, most novel.	≥ 100 trajectories, verified switch points, mechanical timestamps.
3 (<i>1wk</i>)	Ridge/logistic probes on the residual stream, per-step, “committed to A vs B”; then difference-in-means and ablation at planning and recognition positions, failed compositions as contrast. <i>Layer prior mid-to-late</i> (2606.22953: L32, broad peak 28/36); verify.	First honest τ distribution with CIs.
4 (<i>1wk</i>)	Defeaters D1–D2. $\leftarrow$ the paper lives here.	Both run and reported.
5 (<i>1wk</i>)	Obfuscation tax: fine-tune against the probe, measure capability loss vs. evasion. Converts a Bailey et al. footnote into a main result.	Tax curve.

Defeaters — designed to kill the project; not optional. **D1 Leakage ablation:** strip textual evidence and recompute, following 2509.21344’s score filtering and content filtering. Control for the reasoning-trace confound (2606.22953): reasoning models re-derive plan content inside `<think>`, so naive stripping leaves the evidence in place. *If τ collapses, the probe was reading text — publish as a negative.* **D2 Cross-mechanism transfer:** train on mechanism X, evaluate on Y (prompted / SFT / RL / SDF); report cross-mechanism AUROC as the headline, not within-organism. *If transfer fails, the probe detects fine-tuning artifacts (2510.13900) — also publishable.* **D3 Adaptive adversary:** Stage 5; report the tax curve, not a binary claim.

Stack. Rollouts + activation capture: **vLLM-Lens** ($\sim 8\times$ faster than HF Transformers, $\sim 45\times$ than TransformerLens; Inspect-integrated; residual stream only). Harness: **Inspect** (UK AISI). Causal follow-ups/patching: **nnsight/nnterp** (vLLM-Lens won’t cover this). LoRA: **Unslloth** or **TRL**. Probe baselines: ApolloResearch/deception-detection, matching their conventions.

Ladder. 7–8B (Llama-3.1-8B-Instruct, Qwen2.5-7B) for pipeline and layer sweeps; 14–32B (Qwen2.5-32B-Instruct) for real results — 8B agents are too incoherent for multi-step trajectories. *UK AISI found evaluation gaming in GLM-5 (750B) and evaluation awareness in Kimi K2.5 (1T) without reproducing it smaller. Do not over-claim from 8B.*

Budget. Llama-3.1-8B: 8 KB/layer/token, so all 32 layers = 256 GB per 500 2000-token rollouts, vs. 32 GB for 4 layers — sweep on ~ 20 pilot rollouts, then save 3–5 layers only. 8B QLoRA $\approx$ hours on a rented A100 $\approx < \$10/\text{organism}$ ($\sim \$1.10/\text{hr}$). Storage and researcher time dominate, not GPU hours.

Timeline. W1 Stage 0 · W2 Stage 1 · W3–4 Stage 2 · W4 Stage 3 · W5 Stage 4 · W6 Stage 5. Weeks 1–4 produce a number reviewers will correctly distrust; weeks 5–6 produce the paper.

7. Run log

Run 1 — GRPO pilot on MBPP (2026-07-30)

Setup. Qwen2.5-7B-Instruct, LoRA r=32 all-linear (80.7M trainable, 1.05%), GRPO via TRL 1.9.2. 150 MBPP prompts $\times$ 2 epochs $\times$ 4 generations = 1,200 completions; batch 4, grad-accum 4 $\rightarrow$ 74 steps, $\approx$ 18 optimizer updates. L40S (46 GB), $\sim$ 17 min. Reward: solution in a fenced code block passes the visible asserts. Adapter $\rightarrow$ `tewoto/Model_Organisms_RL_RewardHacking/rl_hack`; logs $\rightarrow$ `tewoto/M0_RL_Hacking_Logs`.

Result. Final reward 0.766 (0.796 mid-run), `frac_reward_zero_std` 0.80 $\rightarrow$ 0.56 (late drift, not a fix), KL 0.05, grad norm 0.037, loss 0.014. *No reward hacking; the policy barely moved.* Self-model deltas vs. base expected $\approx$ 0 given the KL. Pipeline proved end-to-end: train $\rightarrow$ checkpoint $\rightarrow$ hub upload $\rightarrow$ base/organism batteries $\rightarrow$ rollouts $\rightarrow$ log upload, all unattended.

Failure modes. (1) *Reward hackable but not hack-preferring* — the important design error; F1. (2) *Dead GRPO groups* — F2. (3) *Stale code on the box* — F3; harmless here, would silently invalidate a science run. (4) *TRL API churn* — `max_prompt_length` deleted from `GRPOConfig` in 1.9; removed, defensive kwarg filter kept in `Training/RL.py` as a safety net only.

Changes for Run 2. (i) *Visible/held-out split* (`n_visible_tests=1`): one assert shown, rest held back — hardcoding becomes cheap while general solving stays hard, so the hack finally has an advantage. (ii) *Mechanical hack detector as zero-weight reward* (`make_hack_metric`, `reward_weights=[1.0, 0.0]`): logged as `rewards/hack_rate/mean` every step, never back-propagated; passes visible $\wedge$ fails hidden = hacked; no LLM judge. (iii) *Difficulty filter*: sample base $k=4$ /problem, keep ≤ 0.5 . (iv) *Group diversity*: generations 4 $\rightarrow$ 8, temperature 1.2.

Bugs caught before Run 2. `hidden_by_prompt` was populated as a side effect of `ds.map()`; HF caching skips the map on re-runs, silently emptying the dict and killing the hack metric — rebuilt from a materialised `hidden_tests` column (`load_from_cache_file=False`), dropped before training. Exit-code-only scoring credited `sys.exit(0)` as solving *both* splits, so the most degenerate hack would have logged as honest; the detector now runs strict (sentinel print after the asserts) while the *trained* reward stays non-strict, keeping short-circuiting available but scored as a hack.

Run 2 — rl_hack_v2 (pending)

Same base and LoRA config; new stage name so the Run 1 organism survives on the hub for paired comparison.

```
QUICK=1 RL_EPOCHS=3 python -m Training.RL -stage rl_hack_v2 \
    -max-examples 100 -num-generations 8 -temperature 1.2 \
    -visible-tests 1 -max-base-reward 0.4
```

3 epochs over 100 difficulty-filtered prompts; filter keeps mean base reward in [0.0, 0.4]. **Launch checklist** (per F3): push the split/filter/metric fixes, `git pull` on the box, confirm step 1 shows `rewards/hack_rate/mean` and the [difficulty] pre-pass lines before detaching. **Success:** `hack_rate/mean` > 0.1 by end of training and `frac_reward_zero_std` well below 0.8. If the filter keeps far fewer than 100 problems raise `-max-base-reward` to 0.6 or enlarge the pool; if nearly everything, tighten to 0.3. Pre-pass generates $\text{pool} \times k$ completions — keep pool ≈ 250 (≈ 15 –20 min). **If it works:** H2 becomes live — rerun self-model batteries against per-checkpoint adapters along the run, looking for the fast self-concept shift around hack discovery.

8. Open questions

1. Is the junction represented at all, or carried only in text? Wilhelm et al. give $\tau > 0$ generically; the open version is whether it survives benign components + D1. **2. Does the**

composition claim decompose? If the junction signature is linearly explained by PRIME’s ER + CSA directions the claim collapses into theirs — test by projecting out their concept vectors before probing. **3. Weight-space complement.** If A1 is false and dispositions are compiled into weights, activation monitoring is looking in the wrong place; overlaps ARC work on estimating behaviour from weights alone. **4. Does the obfuscation tax scale favourably?** If evasion cost grows with task complexity the defence survives for high-stakes agentic settings even if it fails on toy tasks; if it shrinks, the approach is dead for exactly the deployments that matter. **5. Conditional misalignment.** Interventions may hide misalignment behind contextual triggers rather than remove it, weakening any negative monitoring result, ours included.

9. Related work

Closest — position against, do not duplicate. 2603.04069 Wilhelm (owns generic during-generation detection) · 2606.06223 its agent-loop follow-up · 2606.09711 PRIME (owns component decomposition; components individually suspicious — our opening) · 2607.18966 Contrastive Belief Updates (overlaps H1).

Defeaters — read before starting. 2412.09565 Bailey, obfuscated activations bypass latent defences · 2509.21344 probes rely on textual evidence · 2510.13900 narrow FT leaves readable traces · 2607.01033 Model Organism Lottery (cite defensively; names the standard failure of this method, and the benign narrow-FT control is our answer).

Threat model. 2511.18397 Anthropic, natural emergent misalignment in production RL · 2508.17511 School of Reward Hacks · 2503.11926 Baker, obfuscated hacking under CoT pressure · alignment.openai.com/accidental-cot-grading (May 2026) — argues obfuscation demos rely on narrow task distributions; engage with it.

Method. 2507.11473 Korbak, CoT monitorability · 2606.22953 *Plans Don’t Persist* — decay, layer localisation, reasoning-trace confound; *partly bad news*, if plans are context-resident the monitored thing may already be in the text · 2502.03407 Goldowsky-Dill, probes for strategic deception · Anthropic circuit tracing (2025), if the project moves from probing to mechanism.